

VOL. IV · ISSUE 09 · MAY 2026



PrintLoop

Setup Manual

From folders of code to live on the internet — in eight parts.

FOR

Abdurrahman

Founder · PrintLoop Technologies Ltd.

Lagos, Nigeria · Volume One

Welcome

This manual takes you from the folder of files Claude gave you to a live PrintLoop platform on the internet. Eight parts, totaling roughly 6–10 hours of work, comfortably spread over 2–3 days.

You are not expected to be a programmer. The instructions assume you've never touched a terminal before. They also assume you can read carefully and follow directions — that's all you need.

IMPORTANT — Each part of this manual builds on the previous one. Don't skip ahead. If something fails, the answer is almost always in Appendix A.

Table of contents

- ▶ Part 1 — Get your computer ready
 - ▶ Part 2 — Understand what you're building
 - ▶ Part 3 — Set up the database
 - ▶ Part 4 — Get the backend running on your laptop
 - ▶ Part 5 — Get the frontend running on your laptop
 - ▶ Part 6 — Connect them and test the full flow
 - ▶ Part 7 — Push everything to GitHub
 - ▶ Part 8 — Deploy to the internet
 - ▶ Appendix A — When something goes wrong
 - ▶ Appendix B — Glossary of technical words
 - ▶ Appendix C — Hiring help when you need it
-

01

Get your computer ready



You'll install five free programs. This is one-time work — once it's done, you'll never repeat it on the same computer.

What you're about to install

Software	What it does
Node.js	The runtime that runs JavaScript code on your computer. Both backend and frontend need it.
Git	Tracks changes to your code and lets you push it to GitHub.
VS Code	A code editor — like Microsoft Word, but for code. Free, made by Microsoft.
MySQL	The database that stores users, jobs, payments, etc.
Redis	A fast in-memory store. Used for queues (background jobs) and rate limiting.

Step 1.1 — Install Node.js

1. Open your web browser.
2. Go to <https://nodejs.org>
3. Click the green button labeled "LTS" (Long Term Support). Don't pick "Current".
4. Run the downloaded installer (.msi on Windows or .pkg on Mac). Click Next through everything.
5. Verify it installed correctly:
 - Windows: Press the Windows key, type "cmd", press Enter.

- Mac: Press Command+Space, type "Terminal", press Enter.

In that black window, type these two commands one at a time and press Enter after each:

```
node --version  
npm --version
```

If you see version numbers like v20.18.0 and 10.8.2, you're done. If either shows "command not found", restart your computer and try again.

Step 1.2 — Install Git

1. Go to <https://git-scm.com/downloads>
2. Click your operating system. Download starts.
3. Run the installer. Important: when asked about "Adjusting your PATH environment", choose the middle option.
4. Verify by typing in Terminal: `git --version`. You should see `git version 2.40.something`.

Step 1.3 — Install VS Code

1. Go to <https://code.visualstudio.com>
2. Click Download. Run the installer. All defaults are fine.
3. Open VS Code once after install to confirm it works.

Step 1.4 — Install MySQL

This is the most fiddly install. Take your time.

On Windows

1. Go to <https://dev.mysql.com/downloads/installer/>
2. Download the larger MSI installer (~400MB). The smaller one needs internet during install.
3. Click "No thanks, just start my download" to skip the login prompt.
4. Run the installer. Choose "Server only" when asked which products to install.
5. For authentication method, pick "Use Strong Password Encryption".
6. Set a password and write it down somewhere safe. Avoid spaces and special characters. Suggestion: `printloop-dev-2026`.
7. Click Next through everything else.

On Mac

1. Go to <https://dev.mysql.com/downloads/mysql/>
2. Pick the DMG installer matching your Mac (Apple silicon or Intel).

3. Run the .dmg, double-click the .pkg inside, follow the installer.
4. Set a password and write it down.
5. After install: System Settings → MySQL → click "Start MySQL Server".

Verify MySQL works

In Terminal:

```
mysql -u root -p
```

Enter your password. You'll see a prompt like `mysql>`. Type `exit`; and press Enter to leave.

Step 1.5 — Install Redis

On Windows

Redis isn't natively supported on Windows. Use this archived Microsoft port:

1. Go to <https://github.com/microsoftarchive/redis/releases>
2. Download Redis-x64-3.0.504.msi
3. Run the installer with default settings.
4. Open the Services app (Windows key → "Services"). Find Redis, make sure status is Running.
5. Verify: in cmd type `redis-cli ping` — should return PONG.

On Mac

First install Homebrew if you don't have it:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install and start Redis:

```
brew install redis  
brew services start redis
```

Verify: `redis-cli ping` — should return PONG.

Step 1.6 — Final check

Open Terminal/cmd and run all five commands one after another. All should print version info or PONG without errors:

```
node --version  
git --version  
mysql --version  
redis-cli ping  
code --version
```

CHECKPOINT — If all five succeed, Part 1 is complete. Take a break — Part 2 is short, but Part 3 needs focus.

02

Understand what you're building



Before you touch code, you need a mental picture of how the pieces talk to each other. Spend ten minutes here. It will save you hours later.

The four systems

PrintLoop has four parts that talk to each other:

System	What it does
Frontend (website)	What students see in their browser. Upload, pay, get code. Also includes admin panel and kiosk-tablet screens.
Backend (server)	The brain. Handles payments, generates codes, sends SMS, stores data. Code is in folder 01-backend.
Database (MySQL)	Where all the data lives. Users, jobs, payments, kiosks, audit logs.
Kiosk (Pi + printer)	The physical hardware in print shops. A Raspberry Pi connects to a USB printer and listens for print jobs.

How a single print job flows

1. Student opens printloop.ng on her phone.
2. Uploads essay.pdf and clicks Pay.
3. Frontend sends the file to the backend.
4. Backend stores file in Cloudinary, creates job in database, asks Paystack to charge her card.

5. Paystack confirms payment.
6. Backend generates code M7K3X9 and sends it via SMS through Termii.
7. She walks to a print shop. Kiosk screen says "Enter your code". She types M7K3X9.
8. Kiosk asks the backend: "Is this code valid? If yes, give me the file."
9. Backend checks database, marks job as in-progress, returns the file URL.
10. Pi downloads the file and sends it to the USB printer.
11. Printer prints. She walks away with her essay.

MENTAL MODEL — That's the entire flow. Everything in Parts 3-8 is just making each of those arrows work.

03

Set up the database



Now you'll create the database that stores all the data PrintLoop needs.

Step 3.1 — Open the backend folder

1. Open VS Code.
2. File menu → Open Folder → navigate to PrintLoop-Complete/01-backend/ and click Open.
3. You'll see folders on the left: controllers/, services/, entities/, etc.

Step 3.2 — Open the integrated terminal

VS Code has a built-in terminal so you don't have to switch windows.

- ▶ Press Ctrl+` (the backtick key, usually below Esc) on Windows.
- ▶ Or Control+` on Mac.

A terminal pane opens at the bottom.

Step 3.3 — Create the database

In the terminal:

```
mysql -u root -p
```

Enter the MySQL root password from Step 1.4. You're now inside MySQL. Type these commands one by one:

```
CREATE DATABASE printloop; CREATE USER 'printloop_user'@'localhost'  
IDENTIFIED BY 'change-this-to-a-real-password'; GRANT ALL PRIVILEGES ON  
printloop.* TO 'printloop_user'@'localhost'; FLUSH PRIVILEGES; exit;
```

IMPORTANT — Replace change-this-to-a-real-password with an actual password. Write it down. You'll use it next.

Step 3.4 — Create the configuration file

The backend reads its passwords from a file called `.env`.

1. In VS Code's file list on the left, look for `.env.example`.
2. Right-click it → Copy. Right-click in empty space → Paste. A copy named `".env.example copy"` is created.
3. Right-click that copy → Rename → change to exactly: `.env` (just `.env`, no extension).

Open `.env` and fill in these values:

```
DATABASE_HOST=localhost DATABASE_PORT=3306 DATABASE_USER=printloop_user
DATABASE_PASSWORD=the-password-from-step-3.3 DATABASE_NAME=printloop
REDIS_HOST=localhost REDIS_PORT=6379 JWT_SECRET=any-long-random-string-of-
letters-and-numbers NODE_ENV=development PORT=4000
```

Leave Paystack/Termii/Cloudinary keys empty for now. We'll fill those in Part 8.

Step 3.5 — Install backend dependencies

```
npm install
```

This downloads everything the backend needs. Takes 1-3 minutes. Lots of text scrolls. As long as it finishes without ERROR in red, you're fine.

Step 3.6 — Run migrations

Migrations are scripts that build the database structure (users table, jobs table, etc.).

```
npm run migration:run
```

You should see lines like Migration "CreateKiosksTable..." has been executed successfully.

Step 3.7 — Seed initial data

This adds your first admin user and 12 sample kiosks.

```
npm run seed
```

After this, your database has:

- ▶ 1 admin user (email: admin@printloop.ng, password: admin123 — change this later)
- ▶ 12 sample kiosks
- ▶ 4 pricing rows (₦5/page A4 B&W, ₦25/page A4 color, etc.)

Step 3.8 — Verify

```
mysql -u printloop_user -p printloop
```

Enter the password from Step 3.3. Then:

```
SHOW TABLES;
```

You should see ~15 tables: users, jobs, kiosks, payments, group_sessions, etc. Type exit; to leave.

CHECKPOINT — Part 3 is done.

04

Get the backend running



Step 4.1 — Start the backend

In the terminal (still in 01-backend/):

```
npm run dev
```

You should see lines like:

```
[INFO] Server listening on port 4000 [INFO] Database connected [INFO] Redis
connected [INFO] Worker started: file-cleanup [INFO] Worker started:
watermark
```

If you see red error text, jump to Appendix A.

Step 4.2 — Test it

Open your browser and visit:

```
http://localhost:4000/health
```

You should see JSON response like `{"status":"ok","timestamp":"2026-05-09T..."}`.
The backend is running.

IMPORTANT — Leave this terminal open. Closing it stops the server. To use the terminal for something else, click the + icon at the top right of the terminal panel to open a second one.

05

Get the frontend running



Step 5.1 — Open the frontend folder

Open a SECOND VS Code window — don't close the backend one.

- ▶ File menu → New Window
- ▶ File → Open Folder → PrintLoop-Complete/02-frontend/ → Open

Open a terminal in this new window too (Ctrl+` or Control+`).

Step 5.2 — Install dependencies

```
npm install
```

Step 5.3 — Configure the API URL

Create a file named `.env.local` in the frontend root with:

```
VITE_API_URL=http://localhost:4000
```

Even without this, the frontend uses fake mock data — you can see all the screens. So this is optional until Part 6.

Step 5.4 — Start the frontend

```
npm run dev
```

You'll see VITE v7.3.3 ready in 236 ms with a URL printed below.

Step 5.5 — Open it in your browser

Visit <http://localhost:5173>

Click around to explore:

- ▶ [/index](#) — catalogue of all 28 screens
- ▶ [/app](#) — customer dashboard
- ▶ [/admin](#) — admin console
- ▶ [/kiosk](#) — kiosk screen

CHECKPOINT — Every screen works because it reads from mock data. Part 5 done.

06

Connect frontend to backend



This is where the frontend stops using fake data and starts talking to the backend you set up. This is the longest part.

A full description is in

05-guides-and-checklists/PrintLoop_Complete_Integration_Guide.docx — Part 5 of that guide. The short version:

1. Mock data lives in 02-frontend/src/data/mockData.js.
2. Replace each export with a function that calls the backend instead.
3. We use a library called RTK Query to make this clean.

RECOMMENDED — Because this involves writing real code (not just running commands), we recommend you hire a freelancer for this part — see Appendix C. Budget: 10–15 hours of a Nigerian Node/React developer (~₦150-250k total).

If you want to attempt it yourself, follow the integration guide closely. It names every endpoint and shows exactly which mock function corresponds to which API call.

07

Push everything to GitHub



GitHub is where you store your code in the cloud. Two reasons:

- ▶ Backup — if your laptop dies, your code is safe.
- ▶ Auto-deploy — every time you push, your live site updates.

Step 7.1 — Create a GitHub account

If you don't have one yet:

1. Go to <https://github.com/signup>
2. Use a real email (you'll need to verify).
3. Choose a username — printloop, abdurrahman-printloop, etc.
4. Pick the Free plan.

Step 7.2 — Push the frontend

The frontend folder is already a git repo with a clean commit on main.

1. Go to <https://github.com/new>
2. Repository name: printloop-frontend
3. Set it to Private.
4. Do NOT check any of the "Add README/license/.gitignore" boxes.
5. Click Create repository.
6. On the next page, copy the "push an existing repository" snippet. Paste in your terminal.

It looks like:

```
git remote add origin https://github.com/YOUR_USERNAME/printloop-frontend.git
git branch -M main git push -u origin main
```

It'll ask for your username and a password. GitHub no longer accepts passwords here — you need a Personal Access Token:

7. Go to <https://github.com/settings/tokens>
8. Generate new token (classic).
9. Name: printloop-laptop
10. Scope: check repo
11. Click Generate. Copy the token immediately — it shows only once.
12. Paste this token where it asks for password in the terminal.

Refresh your GitHub repo page — you should see all your files.

Step 7.3 — Push the backend

The backend doesn't have git yet. Initialize it:

```
cd /path/to/PrintLoop-Complete/01-backend git init -b main git add . git
commit -m "Initial backend code"
```

Create another empty private repo on GitHub called printloop-backend (don't add files at creation), then push:

```
git remote add origin https://github.com/YOUR_USERNAME/printloop-backend.git
git push -u origin main
```

Use the same Personal Access Token. Part 7 done.

08

Deploy to the internet



Time to make it real. Frontend goes to Vercel (free). Backend goes to Railway (~\$5-10/month for low traffic).

Step 8.1 — Deploy frontend to Vercel

1. Go to <https://vercel.com/signup>
2. Click Continue with GitHub. Authorize Vercel.
3. From dashboard: Add New → Project.
4. Find printloop-frontend in the list, click Import.
5. Vercel auto-detects Vite. Don't change anything.
6. Click Deploy. Wait 1-2 minutes.
7. You'll get a URL like printloop-frontend-abc123.vercel.app. Open it.

To use a custom domain like printloop.ng:

- ▶ Buy the domain (Namecheap or DomainKing.ng, ~¥15k/year).
- ▶ In Vercel: Settings → Domains → Add your domain.
- ▶ Follow Vercel's DNS instructions at your registrar.

Step 8.2 — Deploy backend to Railway

1. Go to <https://railway.app>, sign in with GitHub.
2. New Project → Deploy from GitHub Repo → printloop-backend.
3. Railway will start trying to deploy. It will fail (we need a database first).
4. In your project click + New → Database → MySQL. Wait for provision.
5. Click + New → Database → Redis. Same.
6. Click on your backend service → Variables tab.
7. Set these environment variables:

```

DATABASE_HOST=${{MySQL.MYSQLHOST}} DATABASE_PORT=${{MySQL.MYSQLPORT}}
DATABASE_USER=${{MySQL.MYSQLUSER}} DATABASE_PASSWORD=${{MySQL.MYSQLPASSWORD}}
DATABASE_NAME=${{MySQL.MYSQLDATABASE}} REDIS_HOST=${{Redis.REDISHOST}}
REDIS_PORT=${{Redis.REDISPORT}} REDIS_PASSWORD=${{Redis.REDISPASSWORD}}
JWT_SECRET=any-long-random-string NODE_ENV=production PORT=4000

```

8. Click Deploy at the top. Should now succeed.
9. Settings → Networking → Generate Domain. You'll get a URL.
10. Test: visit <https://your-backend.up.railway.app/health> — should show `{"status": "ok"}`.

Step 8.3 — Run migrations on Railway

- ▶ Service → Settings → Deploy tab → run a one-off command.
- ▶ First: `npm run migration:run`
- ▶ Then: `npm run seed`

Step 8.4 — Tell the frontend where the backend lives

1. In Vercel: Project Settings → Environment Variables.
2. Add: `VITE_API_URL = https://your-backend.up.railway.app`
3. Vercel → Deployments → three-dot menu on latest → Redeploy.

Step 8.5 — Get your real API keys

Plug in real services. Each takes 10-30 minutes.

Service	What for	Where
Paystack	Payments	https://dashboard.paystack.com/#/signup
Termii	SMS	https://termii.com/signup
Cloudinary	File storage	https://cloudinary.com/users/register/free
Sentry	Error tracking	https://sentry.io/signup/

For each:

1. Sign up.
2. Find the API keys section in their dashboard.
3. Copy the keys.
4. Add as environment variables in Railway.

5. Redeploy the backend.

CHECKPOINT — Once Step 8.5 is done, your full PrintLoop platform is live. Customers can sign up, pay, and print. Congratulations.

0A

When something goes wrong



"Command not found"

Whatever you typed, the computer doesn't know that command. Either:

- ▶ You typed it wrong (check spelling).
- ▶ You haven't installed that program (go back to Part 1).
- ▶ Your terminal hasn't picked up the new install — restart your computer.

"EADDRINUSE" or "port 4000 is already in use"

Something else is using port 4000.

- ▶ The backend is already running in another window — find and close it.
- ▶ Edit `.env`, change `PORT=4000` to `PORT=4001`.

"ECONNREFUSED" when backend connects to MySQL

MySQL isn't running.

- ▶ Windows: Services app → MySQL → Start.
- ▶ Mac: System Settings → MySQL → Start MySQL Server.

"Access denied for user"

Your `.env` has the wrong password. Open `.env`, double-check `DATABASE_PASSWORD` matches what you set in Step 3.3.

"Cannot find module ..."

You forgot to run npm install. Run it.

Frontend shows a blank white page

Open browser DevTools (F12 on Windows, Option+Cmd+I on Mac), look at the Console tab. The error tells you what's wrong. Most common: the frontend can't reach the backend. Make sure both are running.

"git push" rejects you

GitHub stopped accepting passwords. You need a Personal Access Token. See Step 7.2.

Don't know how to fix it

1. Copy the EXACT error and paste it into Google. Add "node.js" or "vite" to the search.
 2. Copy the error and ask Claude (this AI). Paste BOTH the error and the command you ran.
 3. Hire a freelancer (Appendix C).
-

OB

Glossary



Term	Meaning
Backend	Server-side code. Receives requests, talks to database. In folder 01-backend/.
Frontend	Website code. What users see in their browser. In folder 02-frontend/.
Database	Where data is permanently stored. We use MySQL.
Redis	Like a database, but faster and forgets when restarted. Used for queues and rate limiting.
API	A way for two programs to talk. Frontend calls the backend's API.
Endpoint	A single URL on the backend, like /jobs or /payments.
Migration	Script that changes the database structure (creates tables, adds columns).
Seed	Initial data inserted into the database (default admin, sample kiosks).
Environment variable	A setting stored in a .env file, outside the code. Passwords go here.
Repository (repo)	A folder of code tracked by Git.
Commit	A saved snapshot of code changes.
Push	Uploading local commits to GitHub.
Deploy	Getting code running on a server (not your laptop).
JWT	A token that says "this user is logged in".

Webhook	When an external service (like Paystack) calls your backend to notify of an event.
---------	--



Hiring help



Some parts of this manual are genuinely hard. Hiring is not failure; it's smart.

When to hire

- ▶ Part 6 (frontend ↔ backend integration) — best ROI from a freelancer. ~10-15 hours.
- ▶ Custom branding / design tweaks — if you want a designer to refine the editorial look.
- ▶ Kiosk physical setup — when you're ready to set up the first Pi-and-printer combo.

Where to find Nigerian freelancers

Platform	Notes
Upwork	Global marketplace. Well-vetted. Expect \$20-50/hr USD.
Toptal	Premium tier. \$60-100/hr USD.
X (Twitter)	Search "Nigerian developer" or "Lagos developer". Cheaper, less vetted.
Andela alumni network	Top-tier Nigerian engineers. Usually too expensive for small projects.
Tunga / Genesys / similar	Small agencies. They handle management for you.

How to brief a freelancer

Send them:

1. The relevant folder (01-backend/ or 02-frontend/).
2. The relevant guide from 05-guides-and-checklists/.
3. The specific tasks. Don't say "build PrintLoop" — say "wire the print job creation endpoint to the frontend's PrintFlow.jsx".

Sample brief for Part 6

BRIEF TEMPLATE — I have a React frontend that uses mocked data, and a Node/Express/TypeORM backend. I need them connected via RTK Query. Replace each export in src/data/mockData.js with an RTK Query endpoint. Add an auth slice that stores the JWT and adds it to requests. Wire the 4-step print flow: create job → upload file → confirm payment → poll status. Hook up admin pages. Backend documented in 05-guides-and-checklists/PrintLoop_Complete_Integration_Guide.docx. Estimate: 10-15 hours. Budget: ₦150-250k.



End of manual.

Good luck, Abdurrahman. You've got this.